

**COMPSCI 701:**  
**Creating Maintainable Software**  
**Lecture 7.2: More Design Patterns**

Ewan Tempero, Yu-Cheng Tu  
School of Computer Science

# Agenda

- Agenda
- Change Ratio
- More Design Patterns
- Command
- Command and Alterability
- Adapter
- Observer
- Key Points

- Admin
  - Wednesday: Strategy and Visitor Design Patterns
- More Design Patterns
- Command
- Observer

- Agenda
- **Change Ratio**
- More Design Patterns
- Command
- Command and Alterability
- Adapter
- Observer
- Key Points

## Change Ratio

- There isn't a model for alterability so we will create our own — **Change Ratio** ( $CR = \frac{C}{N}$ )
- For a given change case with impact  $I$  ...
  - Determine the number of “classes” (files) of the existing design ( $N$ )
  - Determine the number of those classes (files) that need to be changed while implementing the change case ( $C$ )
  - Difficult to do in practice, but does provide a useful heuristic
  - A change case with low impact that requires almost every class to change (high  $C$ ) is considered low alterability
  - $C$  does **not** include any new classes created
  - A class is counted as changed whether one expression changes or almost every statement changes
  - What happens when we delete a class or refactor a class into two new classes?

- Agenda
- Change Ratio
- More Design Patterns
- Command
- Command and Alterability
- Adapter
- Observer
- Key Points

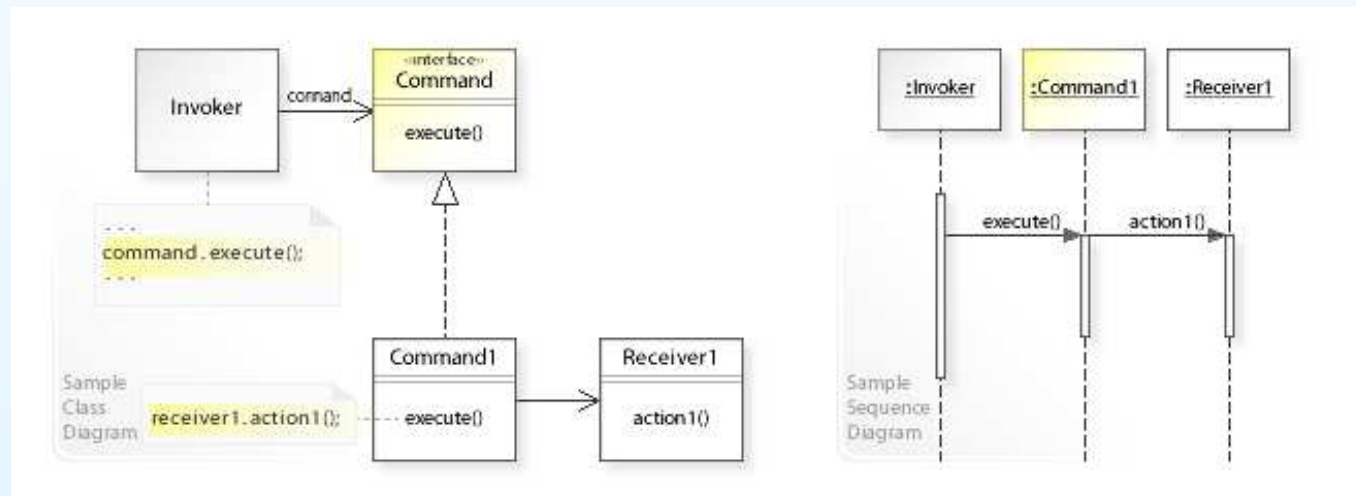
## More Design Patterns

- **Composite** — Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.
- **Command** — Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.
- **Observer** — Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Adapter** — Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.
- **Strategy** — Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Decorator** — Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.
- **Factory Method** — Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
- **Template Method** — Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

# Command Pattern

- Agenda
- Change Ratio
- More Design Patterns
- **Command**
- Command and Alterability
- Adapter
- Observer
- Key Points

- Intent: “Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations”
- Problem: decouple invoker of an operation from the entity that conducts the operation
- Context: decouple in both time and space
- Solution: 4 elements
  - Client — makes the initial request for the need for the command
  - Command — what is to be invoked
  - Invoker — what actually does the invocation
  - Receiver — what the command is actually invoked on



(w3sDesign.com)

- Agenda
- Change Ratio
- More Design Patterns
- **Command**
- Command and Alterability
- Adapter
- Observer
- Key Points

## Example uses

- GUI action implementations (generally, separating what the GUI action does from the GUI itself)
- Delaying when commands are executing
- Recording commands that are executed
- Changing the location where commands are executed

## Filesystem example

- Agenda
- Change Ratio
- More Design Patterns
- **Command**
- Command and Alterability
- Adapter
- Observer
- Key Points

```
private void browse() {
    _root = new Directory("root", "drwx-----");
    _current = _root;
    while (true) {
        System.out.println("L>ist all files");
        System.out.println("New D>irectory");
        System.out.println("New P>lainfile");
        System.out.println("Q>uit");
        String cmd = _keyboard.next();
        if (cmd.equals("L")) {
            printFile(_root);
        } else if (cmd.equals("D")) {
            System.out.print("Enter new directory name:");
            String newDirName = _keyboard.next();
            Directory newDir = new Directory(newDirName, "drwx-----");
            _root.addFile(newDir);
        } else if (cmd.equals("P")) {
            System.out.print("Enter new plain file name:");
            String newFileName = _keyboard.next();
            PlainFile newPlainFile = new PlainFile(newFileName);
            _root.addFile(newPlainFile);
        } else if (cmd.equals("Q")) {
            break;
        }
    }
}
```

- Change case: Add “change directory”

- Agenda
- Change Ratio
- More Design Patterns
- **Command**
- Command and Alterability
- Adapter
- Observer
- Key Points

## Filesystem example

```
private void browse() {
    _current = new Directory("root", "drwx-----");
    while (true) {
        String cmd = getUserOperation();
        if (cmd.equals("L")) {
            printFile();
        } else if (cmd.equals("D")) {
            newDirectory();
        } else if (cmd.equals("P")) {
            newPlainFile();
        } else if (cmd.equals("C")) {
            changeCurrent();
        } else if (cmd.equals("Q")) {
            break;
        }
    }
}
```

- Better but how many other operations do we need?

## Filesystem: Introducing Command

- Agenda
- Change Ratio
- More Design Patterns
- **Command**
- Command and Alterability
- Adapter
- Observer
- Key Points

```
public interface FileOperation {
    void execute();
}

public class ListFiles implements FileOperation {
    private Browser _browser;
    public ListFiles(Browser browser) {
        _browser = browser;
    }
    public void execute() {
        _browser.printFile();
    }
}

public class ChangeCurrent implements FileOperation {
    private Browser _browser;
    public ChangeCurrent(Browser browser) {
        _browser = browser;
    }
    public void execute() {
        _browser.changeCurrentFile();
    }
}
```

- ...and similar classes for other user operations

## Filesystem: Introducing Command

- Agenda
- Change Ratio
- More Design Patterns
- **Command**
- Command and Alterability
- Adapter
- Observer
- Key Points

```
private void browse() {
    _root = new Directory("root", "drwx-----");
    _current = _root;
    while (true) {
        String cmd = getUserOperation();
        FileOperation op = null;
        if (cmd.equals("L")) {
            op = new ListFiles(this);
        } else if (cmd.equals("D")) {
            op = new NewDirectory(this);
        } else if (cmd.equals("P")) {
            op = new NewPlainFile(this);
        } else if (cmd.equals("C")) {
            op = new ChangeCurrent(this);
        } else if (cmd.equals("Q")) {
            break;
        }
        op.execute();
    }
}
```

- `execute()` is called in only one place

- Agenda
- Change Ratio
- More Design Patterns
- **Command**
- Command and Alterability
- Adapter
- Observer
- Key Points

## Filesystem: Introducing Command

```
private void browse() {  
    _root = new Directory("root", "drwx-----");  
    _current = _root;  
    while (true) {  
        String cmd = getUserOperation();  
        FileOperation op = determineFileOperation(cmd);  
        if (op == null) {  
            break;  
        }  
        op.execute();  
    }  
}
```

- This will **never** need to change (in principle)
- We could also move the keyboard input and `FileSystem` methods to another class.

## Command and Alterability

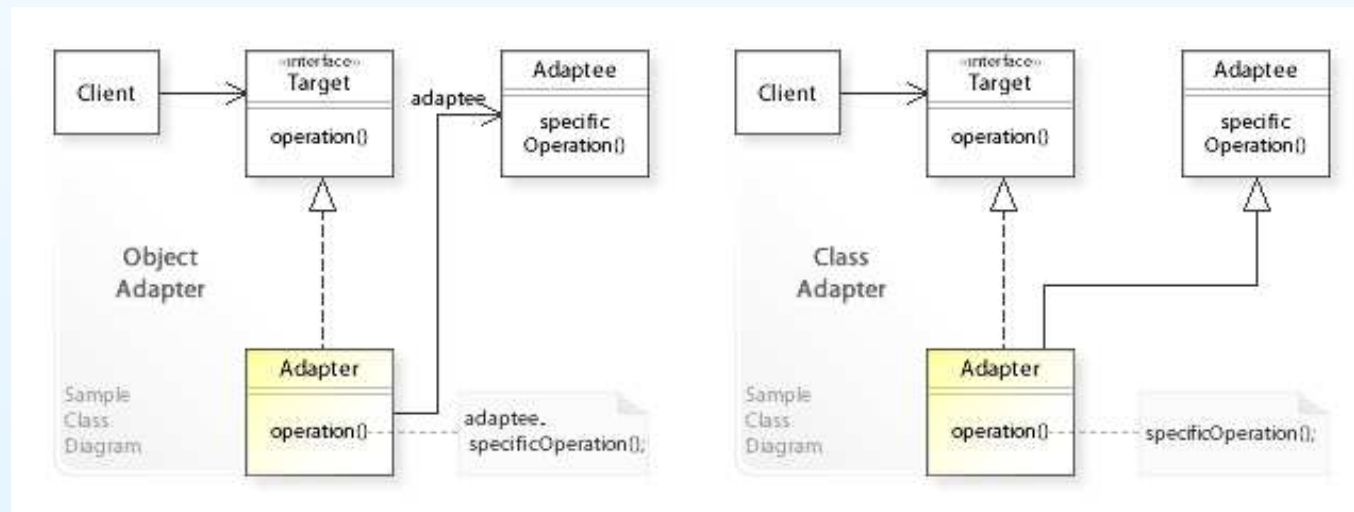
- Agenda
- Change Ratio
- More Design Patterns
- Command
- **Command and Alterability**
- Adapter
- Observer
- Key Points

- Adding new behaviour requires
  - add concrete commands classes (new code, not counted for alterability)
  - create concrete command objects (integrate new concrete commands, requires changing existing code — should be just one class)
  - possibly one other class needs to change to choose the concrete command object to be created
  - Typically  $C = 2$

# Adapter Pattern

- Agenda
- Change Ratio
- More Design Patterns
- Command
- Command and Alterability
- **Adapter**
- Observer
- Key Points

- Intent: “Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn’t otherwise because of incompatible interfaces.”
- Problem: make classes work together that have incompatible interfaces
- Context: need to work with the interface that clients expect
- Solution: 4 elements
  - Client — interacts with the Target interface
  - Target — defines the domain-specific interface that Client uses
  - Adaptee — defines an existing interface that needs adapting
  - Adapter — adapts the interface of Adaptee to the Target interface



(w3sDesign.com)

- Agenda
- Change Ratio
- More Design Patterns
- Command
- Command and Alterability
- **Adapter**
- Observer
- Key Points

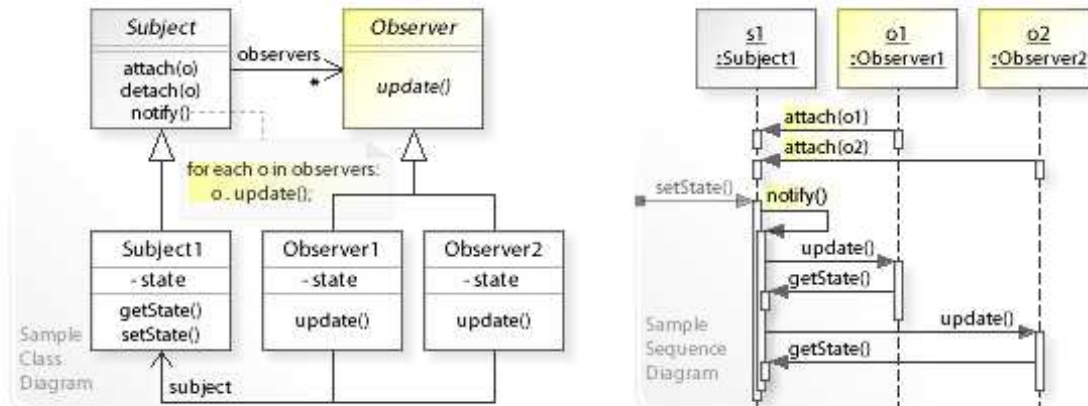
## Example

- An existing file system that prints details about the list of files and directories on the console.
- The system now wants to display the details in a GUI tree. We can consider using using JTree and TreeModel from Java Swing.
- The JTree is the client, and the TreeModel is the target interface. The existing file system is the adaptee.
- A TreeModelAdapter (adapter) is created to provide the information from the file system (adaptee) for the JTree to display. (Object adapter)

# Observer

- Agenda
- Change Ratio
- More Design Patterns
- Command
- Command and Alterability
- Adapter
- **Observer**
- Key Points

- Intent: “Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically”
- The subject should not know anything about the observers. Observers want to be able to be added (and removed) at runtime
- Observers and observed (“subject”)
- Observers want to know whenever the subject changes so they can react (as they want) to the change
- Observers notify the subject of their interest
- The subject may include information about the change in the notification, or not.



- Agenda
- Change Ratio
- More Design Patterns
- Command
- Command and Alterability
- Adapter
- **Observer**
- Key Points

## Example

- A logging system that records events of interest in the system and makes them available (e.g. by writing to a file)
- The logger is the observer, and it registers as such to the various components it wants to log events for
- E.g. the Authentication component manages any user trying to authenticate. The logger is notified for every attempt

- Agenda
- Change Ratio
- More Design Patterns
- Command
- Command and Alterability
- Adapter
- Observer
- Key Points

## Key Points

- Design pattern — captures design experience for solving certain problems in certain contexts
- Most design patterns support Alterability by allowing new functionality to be supplied in new classes, and by reducing the number of existing classes that need to change